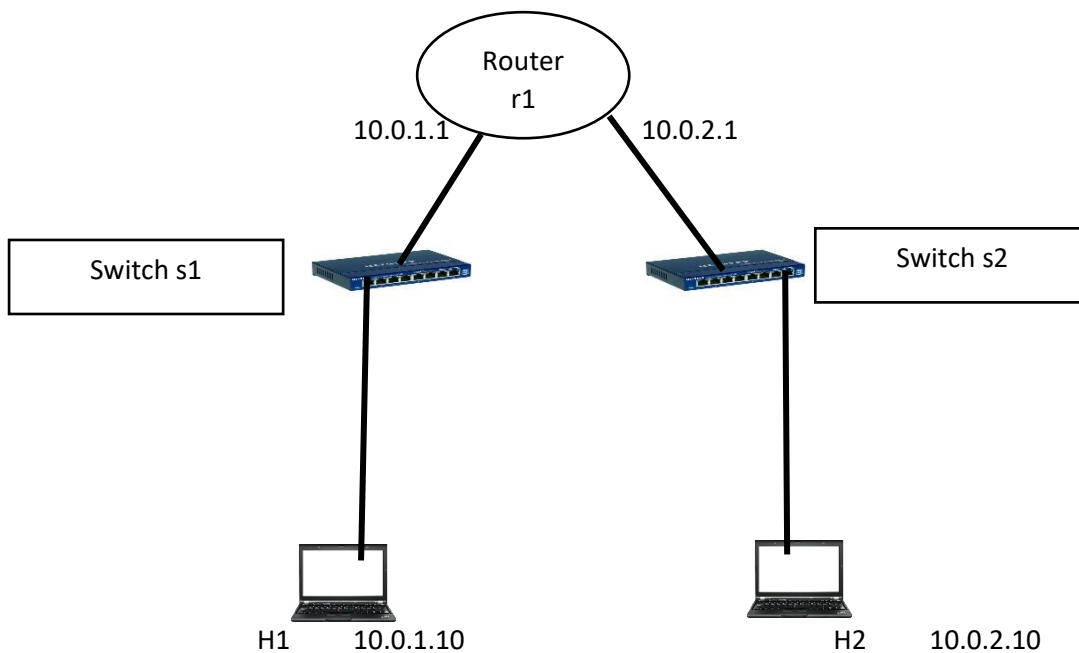


Simple Routing in Mininet

In this part of the lab, we will look at a simple topology including router and switches.

The diagram below will be used, where we have two hosts as h1 and h2, two switches s1 and s2, and a router r1.



Topology creation options:

We can create the above topology using Python code in Mininet. Will focus on this option for this part of the lab.

If you want to create this topology using MiniEdit, you will require packages like Zebra and frr. But we are going with the first option above.

Topology code with router:

Review and understand the following code. Especially, the router and links configuration. Also, look at the policies and flows for traffic routing. The details of these policies will be covered next.

```

from mininet.net import Mininet
from mininet.node import Controller, RemoteController, OVSKernelSwitch,
UserSwitch
from mininet.cli import CLI
from mininet.log import setLogLevel
from mininet.link import Link, TCLink

def topology():
    net = Mininet( controller=RemoteController, link=TCLink,
switch=OVSKernelSwitch )

    # Add hosts and switches
    h1 = net.addHost( 'h1', ip="10.0.1.10/24", mac="00:00:00:00:00:01" )
    h2 = net.addHost( 'h2', ip="10.0.2.10/24", mac="00:00:00:00:00:02" )
    r1 = net.addHost( 'r1' )
    s1 = net.addSwitch( 's1' )
    s2 = net.addSwitch( 's2' )
    c0 = net.addController( 'c0', controller=RemoteController, ip='127.0.0.1',
port=6633 )

    net.addLink( r1, s1 )
    net.addLink( r1, s2 )
    net.addLink( h1, s1 )
    net.addLink( h2, s2 )
    net.build()
    c0.start()
    s1.start( [c0] )
    s2.start( [c0] )
    r1.cmd("ifconfig r1-eth0 0")
    r1.cmd("ifconfig r1-eth1 0")
    r1.cmd("ifconfig r1-eth0 hw ether 00:00:00:00:01:01")
    r1.cmd("ifconfig r1-eth1 hw ether 00:00:00:00:01:02")
    r1.cmd("ip addr add 10.0.1.1/24 brd + dev r1-eth0")
    r1.cmd("ip addr add 10.0.2.1/24 brd + dev r1-eth1")
    r1.cmd("echo 1 > /proc/sys/net/ipv4/ip_forward")
    h1.cmd("ip route add default via 10.0.1.1")
    h2.cmd("ip route add default via 10.0.2.1")
    s1.cmd("ovs-ofctl add-flow s1 priority=1,arp,actions=flood")
    s1.cmd("ovs-ofctl add-flow s1
priority=65535,ip,dl_dst=00:00:00:00:01:01,actions=output:1")
    s1.cmd("ovs-ofctl add-flow s1
priority=10,ip,nw_dst=10.0.1.0/24,actions=output:2")

```

```
s2.cmd("ovs-ofctl add-flow s2 priority=1,arp,actions=flood")
s2.cmd("ovs-ofctl add-flow s2
priority=65535,ip,dst=00:00:00:00:01:02,actions=output:1")
s2.cmd("ovs-ofctl add-flow s2
priority=10,ip,nw_dst=10.0.2.0/24,actions=output:2")

print ("*** Running CLI")
CLI( net )

print ("*** Stopping network")
net.stop()

if __name__ == '__main__':
    setLogLevel( 'info' )
    topology()
```

Running code in Mininet:

You must be logged in as a root user to run Mininet. I have saved this file as RouterLab.py. Use the command below to run the file.

python3 RouterLab.py

```
root@saimvm:/home/saim/mininet/mininet/examples# python3 routerlab.py
Unable to contact the remote controller at 127.0.0.1:6633
*** Configuring hosts
h1 h2 r1
*** Running CLI
*** Starting CLI:
```

At this stage we are not concerned about the controller, so ignore the warning message.

Testing router configuration:

Run the command,

r1 ifconfig to see the interface configuration of the router.

```

mininet> r1 ifconfig
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

r1-eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.1.1 netmask 255.255.255.0 broadcast 10.0.1.255
    inet6 fe80::bcf3:cbff:fe93:fed7 prefixlen 64 scopeid 0x20<link>
    ether 00:00:00:00:01:01 txqueuelen 1000 (Ethernet)
    RX packets 23 bytes 3488 (3.4 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 9 bytes 726 (726.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

r1-eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.2.1 netmask 255.255.255.0 broadcast 10.0.2.255
    inet6 fe80::7c61:9aff:fe78:6c79 prefixlen 64 scopeid 0x20<link>
    ether 00:00:00:00:01:02 txqueuelen 1000 (Ethernet)

```

Use **sh ip route** command to check the default ip routes. If you are using a VM you might see default route of your VM IP address.

```

mininet> sh ip route
default via 10.0.2.2 dev enp0s3 proto dhcp metric 100
10.0.2.0/24 dev enp0s3 proto kernel scope link src 10.0.2.15 metric 100
169.254.0.0/16 dev enp0s3 scope link metric 1000

```

Now try ping both hosts from each other.

h1 ping h2

h2 ping h1

```
mininet> h1 ping h2
PING 10.0.2.10 (10.0.2.10) 56(84) bytes of data.
64 bytes from 10.0.2.10: icmp_seq=1 ttl=63 time=0.854 ms
64 bytes from 10.0.2.10: icmp_seq=2 ttl=63 time=0.138 ms
64 bytes from 10.0.2.10: icmp_seq=3 ttl=63 time=0.141 ms
64 bytes from 10.0.2.10: icmp_seq=4 ttl=63 time=0.097 ms
64 bytes from 10.0.2.10: icmp_seq=5 ttl=63 time=0.139 ms
64 bytes from 10.0.2.10: icmp_seq=6 ttl=63 time=0.162 ms
64 bytes from 10.0.2.10: icmp_seq=7 ttl=63 time=0.143 ms
64 bytes from 10.0.2.10: icmp_seq=8 ttl=63 time=0.137 ms
64 bytes from 10.0.2.10: icmp_seq=9 ttl=63 time=0.155 ms
^C
--- 10.0.2.10 ping statistics ---
9 packets transmitted, 9 received, 0% packet loss, time 8173ms
rtt min/avg/max/mdev = 0.097/0.218/0.854/0.225 ms
mininet> h2 ping h1
PING 10.0.1.10 (10.0.1.10) 56(84) bytes of data.
64 bytes from 10.0.1.10: icmp_seq=1 ttl=63 time=0.076 ms
64 bytes from 10.0.1.10: icmp_seq=2 ttl=63 time=0.142 ms
64 bytes from 10.0.1.10: icmp_seq=3 ttl=63 time=0.140 ms
64 bytes from 10.0.1.10: icmp_seq=4 ttl=63 time=0.192 ms
64 bytes from 10.0.1.10: icmp_seq=5 ttl=63 time=0.137 ms
```

TASK:

Try commenting the lines,

```
#net.addLink( r1, s1 )
```

```
#net.addLink( r1, s2 )
```

Close your routinglab.py file and re-run this file after commenting the above lines and saving your python file.

Try to ping again h2 to h1, and h1 to h2. See what happens.